

Multi-Objective Black-Box Test Case Selection for System Testing

Remo Lachmann
TU Braunschweig
Germany
r.lachmann@tu-bs.de

Michael Felderer
University of Innsbruck
Austria
michael.felderer@uibk.ac.at

Manuel Nieke
TU Braunschweig
Germany
manuel.nieke@tu-bs.de

Sandro Schulze
Otto-von-Guericke-Universität
Magdeburg
Germany
sanschul@iti.cs.uni-magdeburg.de

Christoph Seidl
TU Braunschweig
Germany
c.seidl@tu-bs.de

Ina Schaefer
TU Braunschweig
Germany
i.schaefer@tu-bs.de

ABSTRACT

Testing is a fundamental task to ensure software quality. Regression testing aims to ensure that changes to software do not introduce new failures. As resources are often limited and testing comprises a vast amount of test cases, different regression strategies have been proposed to reduce testing effort by selecting or prioritizing important test cases, e.g., code coverage (to ensure a sufficient testing depth). However, in system testing, source code is often not available creating a black-box system. In this paper, we introduce an automated, multi-objective test case selection technique in black-box systems using genetic algorithms. We define seven different objectives, based on meta-data, allowing a flexible test case selection for a variety of systems. For evaluation, we apply our technique on two different subject systems assessing the feasibility and suitability of our test case selection approach. Results indicate that our approach is applicable based on different data available and is able to outperform random test case selection and retest-all.

CCS CONCEPTS

•Software and its engineering → Software testing and debugging; •Theory of computation → Evolutionary algorithms;

KEYWORDS

Test Case Selection, Search-based Testing, Black-Box Testing, System Testing

ACM Reference format:

Remo Lachmann, Michael Felderer, Manuel Nieke, Sandro Schulze, Christoph Seidl, and Ina Schaefer. 2017. Multi-Objective Black-Box Test Case Selection for System Testing. In *Proceedings of GECCO '17, Berlin, Germany, July 15-19, 2017*, 8 pages. DOI: <http://dx.doi.org/10.1145/3071178.3071189>

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

GECCO '17, Berlin, Germany

© 2017 ACM. 978-1-4503-4920-8/17/07...\$15.00

DOI: <http://dx.doi.org/10.1145/3071178.3071189>

1 INTRODUCTION

Testing is a major task of software engineering [2, 19]. It is performed continuously for each new software version ensuring conformance of the system to its specification after changes have been performed. In particular, it is of importance to ensure that new functionality is correct and that already tested parts of the system do not render erroneous due to introduced changes [29]. Unfortunately, the execution of all test cases is often infeasible due to resource restrictions. *Regression testing* techniques have been introduced to reduce this testing effort [33]. This paper focuses on *test case selection*.

The main goal of test case selection is to find a representative subset of test cases which fulfills testing goals, such as high code coverage, high failure finding potential or high test cases diversity [12, 29, 33]. Most of the existing selection approaches consider source code or test models as input for test case selection and are not multi-objective [4, 12]. However, in system-level testing of component-based software, source code and models are often not available to the tester as components might be provided by external suppliers [30]. Such systems are a *black-box* and tested against their specification, i.e., the requirements defined on system level [28].

In this work, we introduce a novel test case selection approach, based on system-level information, aiming to find test cases that have a high chance to reveal failures. To this end, we introduce a flexible search-based test case selection technique, including risk-related objectives, which is a novelty in search-based testing [13]. In particular, we use a genetic algorithm [24] that aims at optimizing several objectives at once, thus, finding Pareto-optimal test sets.

Contributions:

- We define *seven different black-box objectives* for search-based testing that can be combined freely by users of our technique while current test case selection techniques usually consider only one or two pre-defined objectives as selection criteria. In addition, we consider *risk-based objectives*, which is a novelty in search-based testing and an important aspect for testing especially in safety-critical software systems.
- We evaluate the *feasibility* of our technique using *two different case studies* from academia and real-life industrial

data. They provide different data artifacts, demanding a flexible test case selection approach.

- We show the *suitability* of our technique in terms of precision and recall. It is able to *outperform* a random test case selection approach and retest-all for both systems using a different combination of objectives regarding available data artifacts.

2 RELATED WORK

Engström et al. [12] conducted a survey of test case selection techniques in regression testing. They found a variety of different techniques but black-box test case selection techniques are still sparse. In this work, we apply search-based testing for a flexible test case selection approach, thus, we present related work in this context.

Multi-Objective Search-based Selection. Search-based testing gained some popularity in recent years [26] and, due to its generic nature, has been applied in different contexts such as structural [3] or stress testing [5]. However, many existing search-based approaches focus on white-box systems, in particular, branch coverage [18]. Huang et al. present a history-based prioritization approach using a genetic algorithm [22]. They use the revealed failures with their severity as well as the cost of a test case (i.e., execution time) to formulate a fitness function. Compared to our approach, they only define one fitness function to prioritize test cases instead of test case selection. De Souza et al. [7] define one of the first multi-objective test case selection techniques, based on particle swarm optimization. They define two objectives, maximizing requirements coverage and reducing testing effort. Yoo and Harman [32] present a multi-objective test case selection strategy. As objectives, they introduce past failure revealing information, execution cost and, in contrast to our approach, code coverage. They compare search-based algorithms with a greedy approach where both techniques are able to deliver good results in terms of finding the Pareto-front. They do not consider any other objectives. Mondal et al. [27] define a multi-objective test case selection technique using search-based algorithms. However, they apply testing on source code level, i.e., they propose a white-box testing technique. Their technique tries to maximize both code coverage and test case diversity. According to a recent study [16], search-based testing concepts have not been applied in the context of risk-based testing making our approach one of the first in this direction.

History-based Selection. Ekelund and Engström [10] present a regression test case selection strategy which is based on historical information. Its main focus is in covering test cases which have “flipped” their status since the last test. In contrast to our work, they correlate code and test cases to identify test cases which refer to changes. A prioritization method is proposed by Engström et al. [11] based on a concept by Fazlalizadeh et al. [14], which takes historical test information and a static test case priority amongst other criteria into account. Their technique shows good results in an industrial case study. In contrast to our work, they require data of specific runs and prioritize test cases.

Black-Box Selection and Prioritization Techniques. Herzig et al. [21] define a self-adaptive test case selection technique based on execution costs. The goal of their approach is to skip certain

test cases if they exceed the expected costs of not executing them. While the approach is applicable in black-box testing, it only relies on (predicted) execution costs. Hemmati et al. [20] use a similarity-based approach for test case selection. The fitness of a test set is the sum of all pairwise similarities between test cases. In contrast to our work, they use a model-based approach based on state machines.

Summarizing, we found that no previous related work exists that considers multi-objective testing in the black-box domain nor work which assesses search-based approaches for risk-based testing. We could not empirically compare our technique to other existing techniques as they either prioritize test cases, require different artifacts as input, or tools have not been publicly available for evaluation purposes.

3 BACKGROUND

Test Case Selection. As software is updated regularly throughout the life-cycle, testing is a crucial task to ensure the quality of each software version. *Regression testing* aims at ensuring that previously tested parts have not been influenced by new changes. Different regression testing techniques have been proposed, one being test case selection [33]. The goal is to reduce the number of executed test cases by selecting the most important ones. This requires selection criteria depending on the current testing status and the system under test (SUT). As described in Sec. 2, different selection approaches already exist, mainly differentiated by the data available for testing [12].

In this work, we focus on test case selection for black-box systems. We aim to find a test set that has a high fault detection rate while significantly reducing the number of executed test cases. As knowledge about failure-finding potential is not available a-priori, we introduce a test case selection technique which is solely based on black-box meta-data, which is used as selection heuristics and available in system-level testing.

Search-based Testing and Genetic Algorithms. To automate parts of testing, search-based techniques have received attention in the past. They apply meta-heuristic optimization search techniques, such as *genetic algorithms* (GA), to achieve different test goals. As the approach is very generic, it can be used for different optimizations [18].

GAs can be used to solve optimization problems where the optimum is unknown [24]. Similar to natural evolution, GAs are able to overcome local optima due to crossover and mutation operators, following a “survival of the fittest” approach. GAs operate iteratively on a *population*, which consists of a number of *individuals* being solution candidates to the considered problem. An individual itself consists of smaller parts, referred to as *genes*. Moreover, each individual has a certain *fitness* to solve the problem. The assessment of this fitness is the corner stone of GA, as it indicates the quality of an individual to solve the considered problem. A selection step selects n individuals depending on a chosen *selection operator* [6]. Afterwards, a set of new individuals is generated using *crossover* operators. They recombine two existing individuals creating new offspring. Subsequently, a *mutation* step is applied to individuals in the population to slightly adapt them. This initiates a new iteration, selecting the fittest individuals for the next cycle. If more than one fitness function is to be optimized simultaneously, *Pareto-optimal*

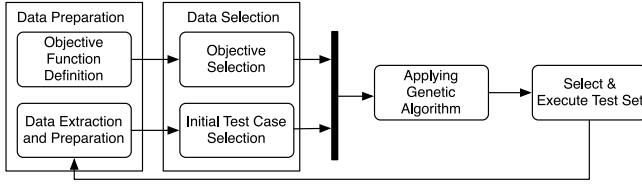


Figure 1: Overview of the Test Case Selection Technique

solutions are selected, i.e., they are not dominated by other solutions in terms of fitness considering all optimization objectives at once [23] Pareto-optimal solutions can not be further improved in one fitness value without decreasing another objective value.

4 CONCEPT

In this section, we describe the general concept of our multi-objective test case selection technique (shown in Fig. 1).

4.1 Data Preparation

First the meta-data used as input for our test case selection technique as well as the objective functions using this data for optimization have to be determined.

Data Extraction and Preparation. As black-box testing comes without code access, we need to extract available data to decide on the importance of test cases. Our technique is flexible in terms of required input data. We are able to handle combinations of the following black-box data as input for test case selection:

Covered Requirements: Each test case tc in the set of all test cases TC covers at least one requirement req defined in the projects set of requirements REQ and each requirement is covered by at least one test case. Requirements are defined in natural language as specification for the system.

Failure Probability: Each requirement is assigned a probability value $fprob : REQ \rightarrow \mathbb{N}$ of its likelihood to fail. This information has to be assigned manually according to risk-based testing techniques [1, 17]. We assume this value to be defined on a fixed nominal scale, e.g., from 1-5.

Business Importance: For each requirement, a business importance value $bv : REQ \rightarrow \mathbb{N}$ is defined based on the functionality it comprises. Similar to the failure probability, this value has to be assigned manually on a fixed scale using risk-based assessment techniques [1, 15].

Revealed Failure: All failures $\mathcal{F} = \{f_1, \dots, f_n\}$ are linked to the respective test cases $TC_f \subseteq TC$, which revealed them. The revealing test cases should be executed to confirm the fix. Additionally, a retest might discover another failure $f' \in \mathcal{F}$ which is only observable after the fix has been applied.

Failure Priority: Each failure $f \in \mathcal{F}$ has a priority $prio : \mathcal{F} \rightarrow \mathbb{N}$ assigned. Test cases linked to very important failures should be retested first. Again, this is defined on a fixed scale.

Average Test Execution Cost: The cost $cost : TC \rightarrow \mathbb{N}$ is defined as the execution time. As it is of high interest to execute as many test cases as possible, we consider execution time as an important testing criterion for the test case selection [7].

Table 1: 6-point Scale for Last Test Case Execution

Scale	1	2	3	4	5	6
Since (Days)	≤ 7	≤ 14	≤ 30	≤ 90	> 90	never

Last Test Execution Time: The last execution of a test case is denoted as $lastExec : TC \rightarrow \{1, \dots, 6\} = \{x \in \mathbb{N} \mid 1 \leq x \leq 6\}$. The age is defined on a fixed 6-point scale based on the number of days since a test case's last execution (cf. Tab. 1). This reduces computational overhead as we do not deal with the actual time in seconds, but an abstract classification of execution times, reducing the value space for this objective. Hence, if a test case has not been executed for a long time, its need to be executed again increases, especially if it has never been executed.

Objective Function Definition. To perform a multi-objective search-based test case selection, certain objectives $O = \{o_1, \dots, o_p\}$ have to be applied to an input of test cases $TC = \{tc_1, \dots, tc_n\}$. The result is a selection of test cases $TS \subseteq TC$, which have a high importance for regression testing. In general, a variety of different test case selection criteria is conceivable [29, 33]. According to the previously described data available in system testing, we define seven objectives to be fulfilled by test sets, which are applicable without implementation knowledge. Due to the search-based nature of our test case selection, a user-defined combination of objectives can be applied to find optimized test sets. We do not normalize the underlying fitness functions as we merely aim at maximizing or minimizing the results.

Objective 1: Minimizing Test Set Size. The intrinsic objective for test case selection is to reduce the number of test cases to be executed. Otherwise, a *retest-all* strategy is performed [29]. The first objective aims at reducing the test set size saving time and resources. Unfortunately, the optimum of this objective leads to $TS = \emptyset$. Hence, we assume that this objective is always used in combination with other objectives to prevent empty test sets. We define the objective to reduce a test set $TS \subseteq TC$ as a fitness function $fit_{size} : \mathcal{P}(TC) \rightarrow \mathbb{R}$, where $\mathcal{P}(TC)$ defines the power set of TC , as follows:

$$fit_{size}(TS) = \frac{|TS|}{|TC|} \quad (1)$$

Objective 2: Maximize Failure Revealing History. To ensure that bugs have been fixed, a test set $TS \subseteq TC$ has a high priority if the according test cases revealed many important failures in the past, distinguishable by their priority (i.e., severity) $prio(f)$. We define the failure priority to be on a fixed 5-point scale, where a priority value of 5 defines the highest importance for a fix and 1 the lowest priority. To this end, we define a fitness function $fit_{history} : \mathcal{P}(TC) \rightarrow \mathbb{R}$ based on the *revealed failures* $f \in F(tc) \in \mathcal{F}$ of a test case tc and the failure priorities $prio(f)$ as follows:

$$fit_{history}(TS) = \sum_{i=1}^{|TS|} \sum_{j=1}^{|F(tc_i)|} \{prio(f_j) \mid f_j \in \mathcal{F}(tc_i)\} \quad (2)$$

Objective 3: Maximize Business Relevance. From a software management perspective, certain requirements might be of special interest, e.g., because they are more important for customers. Based on the business values for n requirements connected to test case tc , we define the fitness function $fit_{BV} : \mathcal{P}(TC) \rightarrow \mathbb{N}$ as:

$$fit_{BV}(TS) = \sum_{j=1}^{|TS|} \sum_{k=1}^{|Req(tc_j)|} \{bv(req_k) | req_k \in Req(tc_j)\} \quad (3)$$

Objective 4: Maximize Failure Probability of Linked Requirements. In a risk-assessment phase, the functionality defined by a requirement has been assigned failure probabilities. Such values are defined on a 3-point scale, where 3 means that the probability of a malfunction is very high. We define a function $fp : REQ \rightarrow \{1, 2, 3\}$, which returns the failure probability value for requirement $req \in REQ$. Next, we define a function $fit_{FP} : \mathcal{P}(TC) \rightarrow \mathbb{N}$ to find test sets which maximize the sum of covered requirements priority values. It is defined as follows:

$$fit_{FP}(TS) = \sum_{j=1}^{|TS|} \sum_{k=1}^{|Req(tc_j)|} \{fp(req_k) | req_k \in Req(tc_j)\} \quad (4)$$

Combining objectives 3 and 4 results in a risk-based selection, as these factors are generally used to compute a risk value for requirements [15]. According to a current survey, risk-based testing has not been considered in search-based test case selection as of now making this a novel approach [13].

Objective 5: Minimize Execution Cost. A very important objective in testing is execution time [7]. The longer it takes for a test suite to be executed, the more likely it is that not all test cases are executed. We assume to know the cost (time in seconds) for executing each test case as function $cost : TC \rightarrow \mathbb{N}$. Never executed test cases yield a value of 0. Hence, if we include such test cases, it leads to selecting only never executed test cases if no other objectives are selected. We define a fitness function $fit_{cost} : \mathcal{P}(TC) \rightarrow \mathbb{N}$ for a test set $TS \subseteq TC$ as follows:

$$fit_{cost}(TS) = \sum_{j=1}^{|TS|} \{cost(tc_j) | tc_j \in TS_i\} \quad (5)$$

Objective 6: Maximize Requirements Coverage. Each test case covers a certain set of requirements. In system testing, it is useful to cover many requirements to reach a high test coverage [31]. Thus, we define a fitness function $fit_{req} : \mathcal{P}(TC) \rightarrow \mathbb{N}$ to maximize the covered requirements as follows:

$$fit_{req}(TS) = \sum_{i=1}^{|TS|} \{|Req(tc_i) | tc_i \in TS\}| \quad (6)$$

Objective 7: Last Test Case Execution. Another goal of testing is to execute every test case at least once. Especially if a test case has never been executed before, it should have a higher likelihood to be selected for testing. As described earlier, we defined the last execution $lastExec : TC \rightarrow \mathbb{N}$ as 6-point scale. Hence, we define the fitness function $fit_{lastExec} : \mathcal{P}(TC) \rightarrow \mathbb{N}$, which should be

maximized as:

$$fit_{lastExec}(TS) = \sum_{j=1}^{|TS|} \{lastExec(tc_j) | tc_j \in TS\} \quad (7)$$

Next, we describe the selection of objectives and test cases, which is used as input for the genetic algorithm.

4.2 Data Selection

Our approach performs a test case selection based on a number of input test cases. A test expert initially selects test cases that are of some importance for the current product status. For example, only executable test cases should be considered, i.e., their covered functionality is present in the current version of the SUT. We require new test cases to be always executed and, thus, they should not be included in the set for test case selection. Similarly, obsolete test cases can be ignored as they are not applicable to the system.

Another important input for the GA are the objectives to be optimized. We can select more than one objective at a time allowing for a flexible multi-objective test case selection. The tester manually has to specify a set of objectives $O_S \subseteq O$. The applicability of the selected objectives depends on the data. In case of this work, a combination of seven objectives can be selected. This makes the approach adaptable to different domains and types of systems or development processes as it is applicable for different types of available black-box data.

To achieve a test case selection, we enforce that the first objective *Minimize Test Set Size* is always selected to avoid test sets consisting of all available test cases. We require at least one other objective to be applied in addition to prevent empty test sets. The proposed test case selection technique is easily extensible as new fitness functions can be designed and applied if the data is available, e.g., source code.

4.3 Applying Genetic Algorithm

After a set of objectives O_S have been selected, we apply the genetic algorithm to perform black-box test case selection. In case of our work, each individual represents a test set, containing a subset of test cases $TS \subseteq TC$, i.e., each gene corresponds to a test case in TC . To apply mutation and crossover, we encode each individual as a bit-vector where each position resembles a test case. A value of 1 represents a selected test case in the current individual and a value of 0 that a test case is not present, i.e., each individual is of the same length. Hence, mutation and crossover operators do not switch the position of genes but are able to change the value of genes adding or removing test cases from the individual.

4.4 Select and Execute Test Set

After Pareto-optimal solutions have been computed, one of the determined test sets has to be selected for testing. However, each of the Pareto-optimal solutions is considered to be an equally good solution to the given optimization problem.

For our technique, an important selection criterion for the "right" test set is the size of the Pareto-optimal test sets, as this has to correspond to the available time by the tester. Other strategies to decide which Pareto-optimal solution to choose depend on the application/domain and require expert knowledge and, thus, are out of scope for this paper.

When executing test cases, each detected failure has to be reported by the tester (e.g., in a test management system). Additionally, if new requirements are added or existing requirements are modified, they have to be updated as well. We recommend to compute new test sets when a new regression testing iteration is due.

5 EVALUATION

In this section, we present the evaluation of our test case selection technique. First, we present our research objectives. Next, we introduce the two subject systems used for evaluation. Afterwards, we explain the metrics and the prototype used to assess our approach. Finally, we present and discuss our results and threats to validity.

5.1 Research Questions

For the evaluation of our test case selection technique, we define the following research questions:

- RQ1:** *Is our technique suitable in terms of the test case selection quality?* For evaluation, we define quality as precision and recall of a test set, i.e., how many failures are found using how many test cases. We compare our technique to a random selection approach and retest-all.
- RQ2:** *Is our technique efficient and flexible enough to deal with incomplete artifacts for testing?* We investigate if our technique is efficient enough to be applied for complex systems. As we apply multi-objective test case selection, we are interested if it in fact can be applied to different systems and for different available test artifacts.

5.2 Subject Systems

We perform the evaluation on two different subject systems, the Body Comfort System (BCS) and an industrial data set. These systems are different in their origin and available data making them suitable to show the flexibility of our test case selection technique.

Body Comfort System. The BCS case study describes an automotive comfort system from academia, comprising several features, e.g., automatic power window, remote control key and a human machine interface [25]. A total of 97 requirements and 128 test cases has been defined for the BCS. Seven failures have been manually seeded, according to previous experiences in students projects. We do not have information about on costs or last executions. Hence, corresponding objectives will not be used for our evaluation.

Real-Life Automotive Data The second subject system is provided by a partner of the automotive industry. The development of this system comprises several years. In total, the test data comprises more than 10 000 requirements, 10 000 test cases and 1 000 defects. All artifacts are described in natural language. Test cases are executed manually. Business relevance and failure probability are not defined for the requirements, thus, which prevents the usage of the corresponding objectives.

5.3 Selection Quality Metrics

We our techniques quality using the *precision*, *recall* and *F-measure* metrics. We assume to have access to the set of failure revealing test cases $TC_F \subseteq TC$ and their corresponding failures.

The function $precision : \mathcal{P}(TC) \times \mathcal{P}(TC) \rightarrow \mathbb{R}$ describes the quality of the selected test set in terms of the ratio of test cases that do reveal a failure compared to the overall number of selected test cases $TS \subseteq TC$. We aim to maximize the number of failure-revealing test cases $tc \in TC_F \subseteq TC$ within the selected test set TS to improve precision. The precision value ranges between 0 and 1 with 1 being the highest available score, which means that all selected test cases are failure revealing. For a set of selected test cases $TS \subseteq TC$, we define *precision* as the percentage of failure revealing test cases TC_F in TS .

$$precision(TS, TC_F) = \frac{|TC_F \cap TS|}{|TS|}$$

We also aim to achieve a high recall of failure revealing test cases $tc_k \in TC_F$, i.e., the test set TS should contain as many of all test cases $tc \in TC_F$ as possible. Similar to *precision*, the function $recall : \mathcal{P}(TC) \times \mathcal{P}(TC) \rightarrow \mathbb{R}$ also returns a value between 0 and 1 with 1 being the best possible score. We define the *recall* for a set of test cases $TS \subseteq TC$ as:

$$recall(TS, TC_F) = \frac{|TC_F \cap TS|}{|TC_F|}$$

Maximizing both precision and recall simultaneously is difficult to achieve as they influence each other. To describe both precision and recall at once, we use the *F-measure* : $\mathbb{R} \times \mathbb{R} \rightarrow \mathbb{R}$. It returns a scoring value between 0 and 1 with the latter being the best value. Based on both, precision and recall, we define the *F-measure* of a test set as follows:

$$F - measure(precision, recall) = 2 \cdot \frac{precision \cdot recall}{precision + recall}$$

5.4 Implementation

We prototyped the technique using the jMETAL Framework [9]. The test data is provided in HP Quality Center¹ (QC). This test management tool stores the requirements, test cases and failures detected for the two subject systems, providing traceability between these artifacts. As GA, we used the jMetal implementation of a standard GA, the NSGA-II [8] algorithm. Tab. 2 contains the list of the operators and their respective configuration used for NSGA-II.

5.5 Methodology

To answer both research questions we perform an evaluation using the objectives defined in Sec. 4.1 in all different combinations on both subject systems. Three objectives applied to both systems are *Minimize Test Set Size*, *Maximize Requirements Coverage* and *Maximize Failure History*. For BCS, we additionally have information on *Business Relevance* and *Failure Probability* of linked requirements. For real-life automotive data, we have actual *Execution Costs* and *Last Test Execution Times*. We fix the minimal test set size objective as it is the "main objective" of test case selection, which is complemented using at least one other objective to prevent empty test sets. Due to performance restrictions in QC when reading and writing data, we preselect a subset of test cases for the real-life automotive data. We chose 577 test cases, which are regularly used as regression test cases, which are linked to 34 failures.

¹QC is part of HP Application Lifecycle Management, url: <http://www8.hp.com/us/en/software-solutions/application-lifecycle-management.html>, date: March 24th 2017

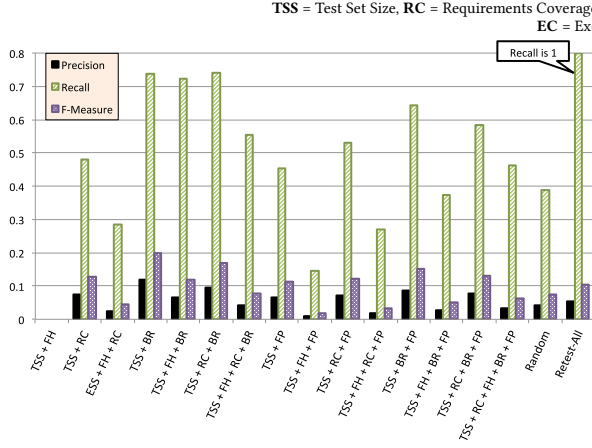


Figure 2: Average Precision, Recall and F-Measure for BCS

Table 2: Applied NSGA-II Operators with Configuration

GA Step	Used Operator	Configuration
Encoding	Binary	Length = $ TC $
Mutation	Bitflip-Mutation	Chance = $\frac{1}{ TC }$
Crossover	HUX-Crossover	Chance = 0.9
Selection	Binary Tournament 2	-

To compute the Pareto optimal solutions, we set the initial population size to the standard value of 100 and perform 5 000 iterations of the genetic algorithm. To reduce the risk of outliers, we repeat the computation for each objective combination *ten times* and present the average of all runs.

We consider test cases $tc \in TC_F$ that find a failure for the computation of precision and recall. Thus, we have to adjust our evaluation for the *failure history* objective. To avoid that the same failures are used for selection and evaluation, we split the set of failures in case the *failure history* objective is used. This is done computing the average failure age. The newer half of failures is used for evaluation, whereas the older set is *used* as meta-data input for the objective.

We compare our results against *retest-all* and *random* selection. For each Pareto-optimal test set retrieved by GA, 1 000 random test sets of the same size are computed in an independent computation. Results of these 1 000 test sets are averaged and, thus, we obtain one normalized random value for each Pareto solution. We do not compare our technique to other approaches, because we either do not have access to the tools or code or the approaches are not applicable to our data artifacts, e.g., due to white-box assumptions.

5.6 Results

In the following, we present the results and answer the two research questions (cf. Sec.5.1).

RQ1: To answer the first research question, we compute the precision and recall as well as the F-measure for each Pareto-optimal

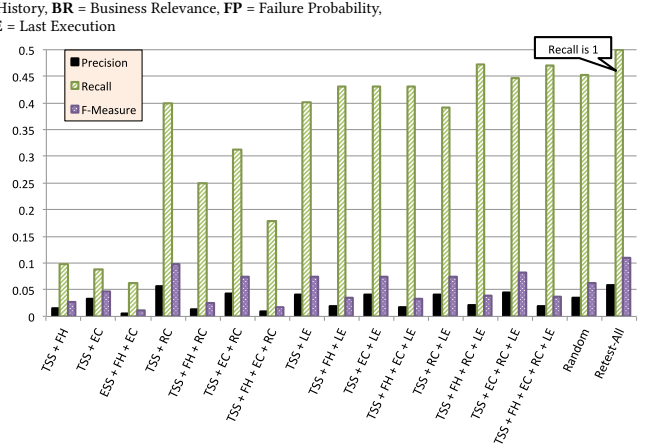


Figure 3: Average Precision, Recall and F-Measure for Real-Life Automotive Data

test set. The results are presented separately for each subject system as we cannot compare them directly due to different data available. In total, we compare 15 different objective combinations for each system. We provide the values for *retest-all* and a random selection as comparison.

BCS. The bar chart in Fig. 2 shows the results of the measured metrics for BCS. It shows the average precision, recall and F-measure for each combination after ten repetitions of the test case selection for 15 combinations as well as the random selection. Retest-all is not included as it inherently achieves a recall of 1, which reduces the diagrams's comprehensibility.

Our data reveal that the combination of *test set size* and *business relevance* outperforms all others in terms of its F-measure achieving a value of about 0.2. This shows the relevance for the consideration of risk-based data in search-based testing, which we introduced in this paper.

The average Pareto-optimal test set size for BCS is 54.5. For our best combination, the precision was nearly 0.12 and the recall was 0.74. In comparison, retest-all achieves a precision of 0.055 and recall of 1, as it selects all test cases, resulting in an F-measure of 0.104. Our best combination reduces the test set size by more than 50% and still improves the F-measure compared to a retest all strategy (i.e., 0.2 compared to 0.104). When we compare the results to the average of 1 000 random selections for each Pareto solution, our technique performs better, too, as the F-measure for the random selection is about 0.07 (cf. Fig. 2).

Real-Life Automotive Data. Similar to BCS, we repeated the evaluation ten times for automotive real-life data. The average results for each combination are shown in Fig. 3. The best combination of objectives, which consists of the *minimize test set size* and *maximize requirements coverage* objectives, achieved an F-measure of 0.1 (precision = 0.06, recall = 0.4).

While a retest-all approach achieves a slightly higher F-measure with 0.11, our test set size is significantly smaller with an average of 201 test cases compared to the total of 577. Hence, we are able to achieve a similar quality as retest-all, while we reduce the number

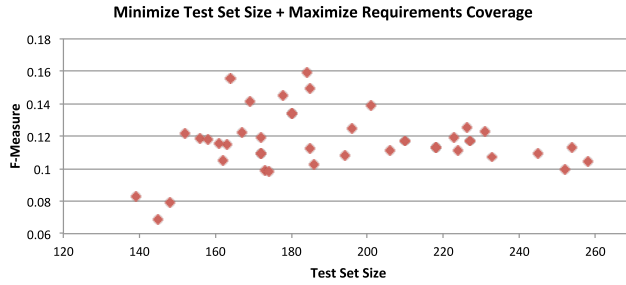


Figure 4: Best Pareto Front for Industrial Data

of executed test cases to 35% of the original number of test cases. This is important, as we are not able to perform all test due to their manual execution. Random selection achieves an average F-measure of 0.06, thus, it is outperformed by our technique (cf. Fig. 3). Moreover, we identified the *execution cost* or *failure history* selected in combination with the minimum test set size to produce the worst results as test cases are only associated with at most one failure.

We present one specific Pareto front in Fig. 4. This front is taken from the best iteration of the real-life automotive data using the combination with the highest average F-measure (cf. Fig. 3). In the diagram in Fig. 4, the dots represent Pareto solutions and they are positioned according to their size (number of test cases) and their F-measure. The size of the Pareto optimal test sets ranges from 140 to 260 test cases. The diagram indicates that there is a sweet spot where the test set size leads to good F-measure results, which is around 180 to 200 test cases (with F-measure being 0.16). Similarly, for BCS, this is at around 40 test cases and with an F-measure of 0.29. In both cases, only about one third of the test cases are used, which indicates an increased effectiveness compared to retest-all.

RQ2: To answer the second research question, we apply our technique on both subject systems using all objectives and their combinations available for each system. We investigate if our technique is flexible enough to handle incomplete artifacts. Hence, we analyze if different input data and perform all different objective combinations for both systems. We analyze if these results are statistically significant, i.e., if different objectives compute a different output. Thus, the technique is flexible in terms of its applicability to incomplete black-box data. In addition, we measure the efficiency of our approach in terms of execution time.

To assess the statistical significance of the results, we perform a *Mann-Whitney U* test between all objective combinations. The test is applied as the distribution of all possible data samples is unknown. In total, we perform 105 pair-wise tests between all available objective combinations for each system. As threshold for the significance tests, we choose the standard alpha-value of 0.05, i.e., two objective combinations do not generate significantly different result sets if their p-value is > 0.05 .

Our results reveal that most of the combination comparisons do return a p-value much lower than 0.05 and, thus, produce significantly different F-measure distributions. However, certain comparisons of results led to higher p-values, shown in Tab. 3. Especially

Table 3: Comparisons with p-value Results greater than 0.05

System	Combination 1	Combination 2	p-V.
BCS	TSS + EC	TSS + RC + BR + FP	0.058
	TSS + FH + BR	TSS + RC + FP	0.511
Industry	TSS + EC	TSS + FH + RC + LE	0.083
	TSS + EC + RC	TSS + EC + LE	0.191
	TSS + LE	TSS + RC + LE	0.348
	TSS + EC + LE	TSS + RC + LE	0.418
	TSS + EC + RC	TSS + RC + LE	0.578
	TSS + EC + RC	TSS + LE	0.832
	TSS + FH	TSS + FH + RC	0.856

Table 4: Average Performance

	BCS	Industry	Random BCS	Random Industry
Input size (TC)	128	577	128	577
Avg. Duration	195 ms	9,200 ms	0.25 ms	3.8 ms

for the real-life automotive data, it seems that the *maximize last execution* objective is similar to the *execution cost* objective when combined with other objectives. In total, due to rather low appearances of tests with p-values > 0.05 , we assume that different objective combinations in most cases produce a statistically significant different output. Hence, a flexible search-based approach is suited to be used for different domains and data input, compared to existing greedy techniques, which are very restricted in their applicability and require certain data to be available.

To assess the applicability of our technique in real-life scenarios, we also measure *efficiency* by means of average computation times. As our results in Tab. 4 reveal, the time for random generation of a test set is very low. For our approach, the times for generating test sets are a) higher and b) differ considerably between BCS (195 ms) and real-life automotive data (9,200 ms). In particular, while real-life data is about four times larger than BCS, it needs about 45 times longer for test case selection. While we cannot entirely explain this difference, we assume that this is due to the different set of objectives (cf. 5.2). In particular, the execution cost objective is defined in seconds, thus, the optimization is more complex compared to the fixed risk-related objectives used for BCS. We claim that our technique is still feasible in a real-life scenario due to overall small computation times.

We do not consider the execution times of test cases for the evaluation of efficiency. This is due to the fact, that our test cases are manually executed, making assumptions about average execution times difficult, e.g., by depending on the person executing the test case. Thus, we only focus on the times for test case selection. Hence, we argue that our approach is applicable in reoccurring regression testing scenarios even with 500 or more test cases.

Our results show that our technique is able to be applied to a different set of systems and data artifacts, producing significantly different output while being efficient in its computation. This makes the technique more flexible than single-objective or greedy approaches as they require an adaption of the fitness function to the SUT or available data.

5.7 Threats to Validity

One *internal* threat to validity is the random nature of GA, which we mitigate by repeating our evaluation *ten times*. In addition, we measured the mean, standard deviation and variance between different runs. As the results reveal only minor differences regarding the standard deviation of F-measures (real-life automotive data: 0.011 - 0.021; BCS: 0.02 - 0.05), we argue that we mitigated effects of GA randomness. Also, the application of many-objective algorithms might be more sufficient in terms of quality when combining more than three objectives.

As *external* threat to validity, one real-life industrial case study does not allow a generalization of results. However, in combination with BCS we are able to show the flexibility of our technique. Hence, both systems combined allow for analyzing both, flexibility and suitability of our test case selection and are sufficient as a first indication of the applicability of our approach.

6 CONCLUSION AND FUTURE WORK

We proposed a novel multi-objective test case selection technique to cope with limited resources in regression testing. The test case selection technique is applicable in black-box system testing where code information is not available and is flexible in terms of necessary data available. We defined seven objectives based on meta-data, which are used as optimization goals for a genetic algorithm computing Pareto-optimal test sets including novel risk-based objectives. Objectives can be combined arbitrarily, which allows for flexibility in testing. In the evaluation, we showed the feasibility and suitability of our technique in two different case studies. Our evaluation results show that the technique is adaptable to individual situations of users and the available data. Furthermore, the evaluation indicates that certain objective combinations achieve better results in terms of precision and recall than a retest-all and random test case selection approach while significantly reducing the number of test cases to be executed. Thus, our technique increases test efficiency and effectiveness.

Due to the generic nature of our multi-objective test case selection technique, future work should comprise the definition of additional objective functions. For example, the approach can be combined with white-box objectives, such as maximizing code coverage or method calls. We assume that this improves the selection quality by introducing a new "view" on the data. More case studies have to be performed to generalize our findings. It would also be desirable to evaluate the approach using all seven objectives at once when relevant data is available. It is worthwhile to examine the possibilities to adapt our approach for test case prioritization.

REFERENCES

- [1] S. Amland. Risk-based testing: Risk analysis fundamentals and metrics for software testing including a financial application case study. *Journal of Systems and Software*, 53(3):287–295, 2000.
- [2] P. Ammann and J. Offutt. *Introduction to Software Testing*. Cambridge University Press, 2008.
- [3] A. Baresel, H. Sthamer, and M. Schmidt. Fitness function design to improve evolutionary structural testing. In *Proc. Genetic and Evol. Comp. Conf.*, pages 1329–1336. Morgan Kaufmann Publishers Inc., 2002.
- [4] S. Biswas, R. Mall, M. Satpathy, and S. Sukumaran. Regression test selection techniques: A survey. *Informatica*, 35(3), 2011.
- [5] L. C. Briand, Y. Labiche, and M. Shousha. Stress testing real-time systems with genetic algorithms. In *Proc. Genetic and Evol. Comp. Conf.*, pages 1021–1028, 2005.
- [6] A. P. Conrad, R. S. Roos, and G. M. Kapfhammer. Empirically studying the role of selection operators during search-based test suite prioritization. In *Proc. Genetic and Evol. Comp. Conf.*, pages 1373–1380, 2010.
- [7] L. De Souza, P. de Miranda, R. Prudencio, and F. de Barros. A multi-objective particle swarm optimization for test case selection based on functional requirements coverage and execution effort. In *Proc. Int'l Conf. Tools Artif. Intel.*, pages 245–252, 2011.
- [8] K. Deb, A. Pratap, S. Agarwal, and T. Meyarivan. A fast and elitist multiobjective genetic algorithm: Nsga-ii. *IEEE Transactions on Evolutionary Computation*, 6(2):182–197, 2002.
- [9] J. J. Durillo and A. J. Nebro. jmetal: A java framework for multi-objective optimization. *Adv. Eng. Softw.*, 42(10):760–771, 2011.
- [10] E. D. Ekelund and E. Engström. Efficient regression testing based on test history: An industrial evaluation. page 9. IEEE Computer Society, 2015.
- [11] E. Engström, P. Runeson, and A. Ljung. Improving regression testing transparency and efficiency with history-based prioritization – an industrial case study. In *Proc. Int'l Conf. Software Testing, Verification, and Validation*, pages 367–376. IEEE, 2011.
- [12] E. Engström, P. Runeson, and M. Skoglund. A systematic review on regression test selection techniques. *Info. and Softw. Techn.*, 52:14–30, 2010.
- [13] G. Erdogan, Y. Li, R. K. Runde, F. Seehusen, and K. Stölen. Approaches for the combined use of risk analysis and testing: A systematic literature review. *Int. J. Softw. Tools Technol. Transf.*, pages 627–642, 2014.
- [14] Y. Fazlalizadeh, A. Khalilian, M. A. Azgomi, and S. Parsa. Prioritizing test cases for resource constraint environments using historical test case performance data. In *Proc. Int'l Conf. on Comp. Science Information Techn.*, pages 190–195. IEEE, 2009.
- [15] M. Felderer, C. Haisjackl, V. Pekar, and R. Breu. A risk assessment framework for software testing. In *Proc. Int'l Symposium Leveraging Applications of Formal Methods, Verification and Validation*, pages 292–308, 2014.
- [16] M. Felderer, C. Haisjackl, V. Pekar, and R. Breu. An exploratory study on risk estimation in risk-based testing approaches. In *Software Quality Days (SWQD) 2015*, pages 32–43, 2015.
- [17] M. Felderer and I. Schieferdecker. A taxonomy of risk-based testing. *Int. J. Softw. Tools Technol. Transf.*, 16(5):559–568, 2014.
- [18] M. Harman and P. McMinn. A theoretical and empirical study of search-based testing: Local, global, and hybrid search. *IEEE Trans. Soft. Eng.*, 36(2):226–247, 2010.
- [19] M. J. Harrold. Testing: A roadmap. In *Proc. Int'l Conf. Software Engineering*, pages 61–72. ACM, 2000.
- [20] H. Hemmati, A. Arcuri, and L. Briand. Achieving scalable model-based testing through test case diversity. *ACM Trans. Soft. Eng. and Methods*, 22(1):6:1–6:42, 2013.
- [21] K. Herzig, M. Greiler, J. Czerwinka, and B. Murphy. The art of testing less without sacrificing quality. In *Proc. Int'l Conf. Software Engineering*. IEEE, 2015. to appear.
- [22] Y.-C. Huang, K.-L. Peng, and C.-Y. Huang. A history-based cost-cognizant test case prioritization technique in regression testing. *J. Sys. and Soft.*, 85(3):626–637, 2012.
- [23] J. Knowles and D. Corne. The pareto archived evolution strategy: a new baseline algorithm for pareto multiobjective optimisation. In *Proc. Congress on Evol. Comp.*, volume 1, page 105, 1999.
- [24] P. Larrañaga, C. M. H. Kuijpers, R. H. Murga, I. Inza, and S. Dizdarevic. Genetic algorithms for the travelling salesman problem: A review of representations and operators. *Art. Int. Rev.*, 13(2):129–170, 1999.
- [25] S. Lity, R. Lachmann, M. Lochau, and I. Schaefer. Delta-oriented software product line test models - the body comfort system case study. Technical report, TU Braunschweig, 2013.
- [26] P. McMinn. Search-based software testing: Past, present and future. In *Proc. Int'l Conf. Softw. Testing, Verification and Validation Workshops*, pages 153–163. IEEE Computer Society, 2011.
- [27] D. Mondal, H. Hemmati, and S. Durocher. Exploring test suite diversification and code coverage in multi-objective test case selection. In *Proc. Int'l Conf. Software Testing, Verification, and Validation*, pages 1–10, 2015.
- [28] S. Nidhra and J. Dondeti. Black box and white box testing techniques—a literature review. 2(2):29–50, 2012.
- [29] G. Rothermel and M. Harrold. Analyzing regression test selection techniques. *IEEE Trans. Soft. Eng.*, 22(8):529–551, 1996.
- [30] S. Shashank, P. Chakka, and D. Kumar. A systematic literature survey of integration testing in component-based software engineering. In *Proc. Int'l Conf. Comp. and Comm. Tech.*, pages 562–568, 2010.
- [31] M. W. Whalen, A. Rajan, M. P. Heimdahl, and S. P. Miller. Coverage metrics for requirements-based testing. In *Proc. Int'l Symposium Software Testing and Analysis*, pages 25–36. ACM, 2006.
- [32] S. Yoo and M. Harman. Pareto efficient multi-objective test case selection. In *Proc. Int'l Symposium Software Testing and Analysis*, pages 140–150. ACM, 2007.
- [33] S. Yoo and M. Harman. Regression testing minimization, selection and prioritization: A survey. *Softw. Test. Verif. Reliab.*, 22(2):67–120, 2007.